Innovative Evaluation Practices for BCSE308P - Computer Networks Lab

Fall Semester 2026-27

NETLAB: NETWORK PERFORMANCE ANALYZER AND SIMULATOR

24BCE0555 ISHITA ANANT

24BCE2989 ANIKET BHAYANA

24BCI0259 DAKSH ARORA

Team 15

B.Tech.

in

Computer Science and Engineering

(Specialization in Information Security: Daksh Arora, 24BCI0259)

School of Computer Science and Engineering



October 2026

ABSTRACT

Textbook formulas for network protocols are usually taken on trust, and the conditions under which they fail are rarely shown. NetLab is a Python network performance analyzer and simulator that studies each protocol at three levels: the textbook formula, our own simulation, and real networking (UDP sockets, Linux network namespaces with tc netem, and a Mininet/FRR testbed). It covers VLSM addressing, RIP and OSPF routing, error detection (parity, checksum, CRC-32, Hamming), the ARQ protocols Stop-and-Wait, Go-Back-N and Selective Repeat, a DNS hierarchy, TCP Reno, CUBIC and BBR congestion control, and a pcap analyzer, all behind a Streamlit dashboard.

Comparing theory with simulation exposed a gap in the textbook Selective Repeat model. It predicts efficiency 1 - p once the window reaches one round trip of frames (1 + 2a), but the measured efficiency stays well below this, because a lost frame blocks the window for a timeout plus a round trip. We propose an RTT-aware window rule, N = 2(1 + 2a), and compare it with the textbook window 1 + 2a over 10 repeated runs per scenario. At a = 5, link efficiency rises from 0.909 ± 0.010 to 0.914 ± 0.008 at 1% loss (+0.6%), from 0.717 ± 0.016 to 0.776 ± 0.009 at 5% loss (+8.2%), and from 0.478 ± 0.008 to 0.590 ± 0.005 at 20% loss (+23.5%). Repeating the scenarios over real UDP sockets gave +0.1%, +7.4% and +24.6%. The benefit grows with the round trip (+1.6% at a = 1 to +18.9% at a = 20) at the cost of doubling the buffer. The rule does not reach the textbook ideal and gives nothing on a loss-free link.

The Mininet and FRR experiments are implemented but were not run, so the routing and Linux TCP results are simulation only. All 94 unit tests pass, and the repository contains the code, results and dashboard screenshots needed to reproduce every figure.

Certificate

This is to certify that the project report titled "NetLab: Network Performance Analyzer and Simulator" is a bonafide record of the work done by Ishita Anant (24BCE0555), Aniket Bhayana (24BCE2989) and Daksh Arora (24BCI0259), Team 15, in partial fulfilment of the requirements of the Innovative Evaluation Practices for BCSE308P - Computer Networks Lab during the Fall Semester 2026-2027.

Name and Signature of the Students

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	2
1.	**INTRODUCTION**	6
	1.1 Background and motivation	6
	1.2 Problem statement	6
	1.3 Objectives	6
	1.4 Scope and limitations	7
	1.5 Mapping to course modules and the lab experiment	7
2.	**Literature Survey**	8
	2.1 Overview of the existing protocol or scheme	8
	2.2 Review of related work	8
	2.3 Research gap and justification of novelty	9
3.	**Proposed System**	10
	3.1 System architecture or topology	10
	3.2 Baseline mechanism description	10
	3.3 Proposed mechanism: algorithm, pseudocode, flowchart	11
	3.4 Novelty statement (what exactly differs from the baseline)	12
	3.5 Assumptions and design decisions	12
4.	**Implementation**	14
	4.1 Tools, platform, versions and hardware	14
	4.2 Simulation or testbed setup	14
	4.3 Code structure and explanation of key modules	15
	4.4 Challenges faced and how they were solved	15
5.	**Experimental Setup and Performance Evaluation**	17
	5.1 Performance metrics and why they were chosen	17
	5.2 Scenarios and parameter values	17
	5.3 Results: tables and graphs, baseline versus proposed	17
	5.4 Analysis and discussion	22
	5.5 Critical evaluation: strengths, weaknesses, conditions where the proposed scheme fails	23
6.	**Conclusion and Future Work**	24
	6.1 Summary of work and key findings	24
	6.2 Limitations	24

6.3 Future enhancements 24

7. References 25

References (IEEE style) 25

Appendix A: Important source code or configuration listings 26

Appendix B: Additional screenshots, packet captures, raw result tables 27

1. INTRODUCTION

1.1 Background and motivation

Computer networks courses teach protocols through closed-form models: the efficiency of Stop-and-Wait is 1/(1+2a), a CRC catches every burst shorter than its width, a TCP flow under random loss settles near the Mathis rate, and OSPF prefers the path with the lowest cost. These statements are true under stated assumptions, but a student rarely sees what happens when an assumption is relaxed, or whether a real network agrees with the formula at all.

NetLab is our attempt to close that gap. Every topic is studied at three levels: the **theory** (the textbook formula), the **simulation** (our own Python implementation of the protocol), and **reality** (real UDP sockets, real Linux network namespaces joined by a veth pair with `tc netem` impairment, and a Mininet + FRR testbed for routing). The three results are plotted on the same axes, so a disagreement becomes something to explain rather than something to hide.

While doing this for Selective Repeat ARQ we found that the textbook model and the measurements disagree in a systematic way. This disagreement is the starting point for the proposed mechanism in Chapter 3.

1.2 Problem statement

The textbook efficiency of Selective Repeat is 1 - p (p is the frame loss probability) whenever the window N is at least one round trip of frames, K = 1 + 2a. Our simulator and real UDP runs show that a window of exactly K stays far below this value as soon as frames are lost. The problem addressed in this project is twofold:

- to build a single, reproducible tool that places theory, simulation and real measurements of the main network protocols side by side; and
- to explain and reduce the Selective Repeat gap by choosing the window from the round-trip delay (N = 2K) instead of the textbook minimum (N = K), and to measure the benefit with repeated runs.

1.3 Objectives

1. Implement from scratch: VLSM addressing, distance-vector and link-state routing, error-detection codes (parity, 2-D parity, Internet checksum, CRC-32, Hamming(7,4)), Stop-and-Wait / Go-Back-N / Selective Repeat, a DNS hierarchy, TCP Reno / CUBIC / BBR, and a pcap analyzer.
2. Run each experiment at the theory, simulation and real levels where the environment allows it, and record the results in a common JSON / CSV / PNG format.

3. Propose and evaluate an RTT-aware Selective Repeat window, reporting mean and standard deviation over repeated runs in three load scenarios.
4. Provide a Streamlit dashboard that lets an examiner change a parameter and see baseline and proposed behaviour live.
5. Provide unit tests and end-to-end scripts so that every claim in this report can be reproduced from the repository.

1.4 Scope and limitations

The work covers the data-link, network, transport and application layers at the protocol-logic level. The limits are stated here so that the results are not over-read:

- The ARQ, TCP and routing evaluations are mostly simulations. The real level for ARQ is UDP over a user-space delay and loss relay on loopback, plus one run across two network namespaces with `tc netem`.
- The Mininet + FRR experiments (RIP versus OSPF on real routers, Linux TCP with iperf3) are implemented and wired into `make testbed`, but they need root and an Ubuntu VM and were **not run** for this report. No result in this document is claimed from them.
- The BBR model is simplified (no ProbeRTT, no loss response), so BBR fairness numbers are not quoted as findings about real BBR.
- A Cisco Packet Tracer lab is described step by step in `docs/phase1/PACKET_TRACER.md`; the `.pkt` file is submitted separately by the team.

1.5 Mapping to course modules and the lab experiment

Course module / layer	Lab experiment in NetLab	Where in the repository
Network addressing, subnetting, VLSM	VLSM planner, topology model, Packet Tracer lab	netlab/addressing, netlab/topology
Routing protocols (RIP, OSPF)	Distance-vector and Dijkstra simulators, count-to-infinity, RIP vs OSPF path trap	netlab/routing, experiments/phase1_*
Data link: error detection	Parity, checksum, CRC-32, Hamming under random, burst and adversarial errors	netlab/reliability/detect.py
Data link: flow and error control	Stop-and-Wait, Go-Back-N, Selective Repeat; **proposed RTT-aware SR window**	netlab/reliability/arq.py, experiments/phase2_sr_window.py
Transport: TCP	Reno, CUBIC, BBR, bufferbloat, fairness	netlab/tcp
Application: DNS	Iterative resolver, caching, NXDOMAIN, resolver hardening	netlab/dns
Network analysis tools	pcap analyzer, Streamlit dashboard	netlab/analyzer, dashboard/app.py

Table 1.1: Mapping of NetLab to course modules

2. LITERATURE SURVEY

2.1 Overview of the existing protocol or scheme

Sliding-window ARQ. A sender keeps up to N unacknowledged frames in flight. In Stop-and-Wait N = 1, so only one frame is outstanding per round trip and the link utilisation is 1/(1+2a), where a is the ratio of propagation delay to frame transmission time. Go-Back-N lets the sender transmit N frames, but the receiver accepts only the next expected frame, so one loss forces the sender to resend the whole window. Selective Repeat buffers out-of-order frames at the receiver and retransmits only the missing frame, each frame having its own timer.

The textbook efficiency model. Stallings [1] gives the utilisation of the three protocols in terms of a and p. For Selective Repeat the result is 1 - p when N >= 1 + 2a and N(1-p)/(1+2a) otherwise. The derivation assumes that the lost frame is the only cost: a lost frame is retransmitted and delivered after one extra round trip, and the rest of the window keeps the link busy meanwhile. The model is the baseline of this project: it states that a window of K = 1 + 2a frames is sufficient for Selective Repeat.

Other protocols studied. Error detection uses parity, the Internet checksum [8] and CRC-32 [9]. Routing uses RIP [10] (hop count, infinity = 16) and OSPF [11] (link-state, cost from bandwidth). DNS follows the iterative resolution of RFC 1034/1035 [12]. TCP congestion control follows RFC 5681 [13], with CUBIC [5] and BBR [6] as modern alternatives.

2.2 Review of related work

S.No	Authors, year	Technique	Metrics and tool	Merits	Gaps
1	Stallings, 2013 [1]	Analytical efficiency of SW, GBN and SR ARQ	Link utilisation as a function of a, p, N; closed-form	Simple, explains why windows matter	Assumes instant loss recovery; no simulation or measurement to check it
2	Kurose and Ross, 2021 [2]	Interactive and simulated treatment of GBN and SR	Throughput and retransmissions in applets, qualitative	Intuition for window behaviour	No sizing rule for N under loss
3	Mathis et al., 1997 [4]	Macroscopic model of TCP Reno	Throughput = 1.22 MSS / (RTT sqrt(p)); simulation	Predicts TCP rate from loss and RTT	Valid only for random, small loss; not BBR
4	Ha, Rhee and Xu, 2008 [5]	CUBIC window growth	Throughput, fairness, ns-2 and testbed	Scales on large bandwidth-delay paths	Fills buffers (bufferbloat)
5	Cardwell et al., 2016 [6]	BBR model-based congestion control	Throughput and RTT on Google WAN	Low queueing delay, loss tolerant	Fairness with loss-based flows is contested
6	Gettys and Nichols, 2012 [7]	Bufferbloat measurement	Latency under load, real home networks	Names a widespread problem	Descriptive; no protocol-level model
7	Lantz, Heller and McKeown, 2010 [3]	Mininet network emulation	Real kernel stack on virtual links	Cheap realistic testbed	A platform, not an analysis of protocol models

S.No	Authors, year	Technique	Metrics and tool	Merits	Gaps
8	Koopman and Chakravarty, 2004 [9]	CRC polynomial selection	Hamming distance, undetected-error rate	Quantifies CRC strength	Analytical; does not compare with checksums on adversarial edits
9	Jain, Chiu and Hawe, 1984 [14]	Fairness index	Jain index over flow throughputs	Single number for fairness	Says nothing about cause

Table 2.1: Literature survey

2.3 Research gap and justification of novelty

The literature gives either an analytical model (items 1, 3, 8) or a measurement platform (items 4 to 7), and the two are seldom put on the same plot. For Selective Repeat in particular, the formula states that K frames are enough, and we found no accompanying measurement that checks this when frames are lost. Our simulator and real UDP runs show that K is not enough, and that the missing factor is close to 2.

The justification of the novelty is therefore (i) the method, a single tool that overlays theory, simulation and real measurement for every protocol, and (ii) the result of that method, the RTT-aware window N = 2K for Selective Repeat, evaluated with repeated runs against the textbook window N = K. We do not claim a new protocol: we claim a measured sizing rule and a way of checking such rules.

3. PROPOSED SYSTEM

3.1 System architecture or topology

Figure 3.1 shows the software architecture. The Streamlit dashboard calls runner functions, which call the protocol libraries (one sub-package per phase). Every library writes its results through a common `Result` class (JSON, CSV and PNG). Figure 3.2 shows the simulated link used for the ARQ experiments: a sender and a receiver joined by a forward channel with rate, delay, loss and reordering, and a lossless reverse channel for acknowledgements.

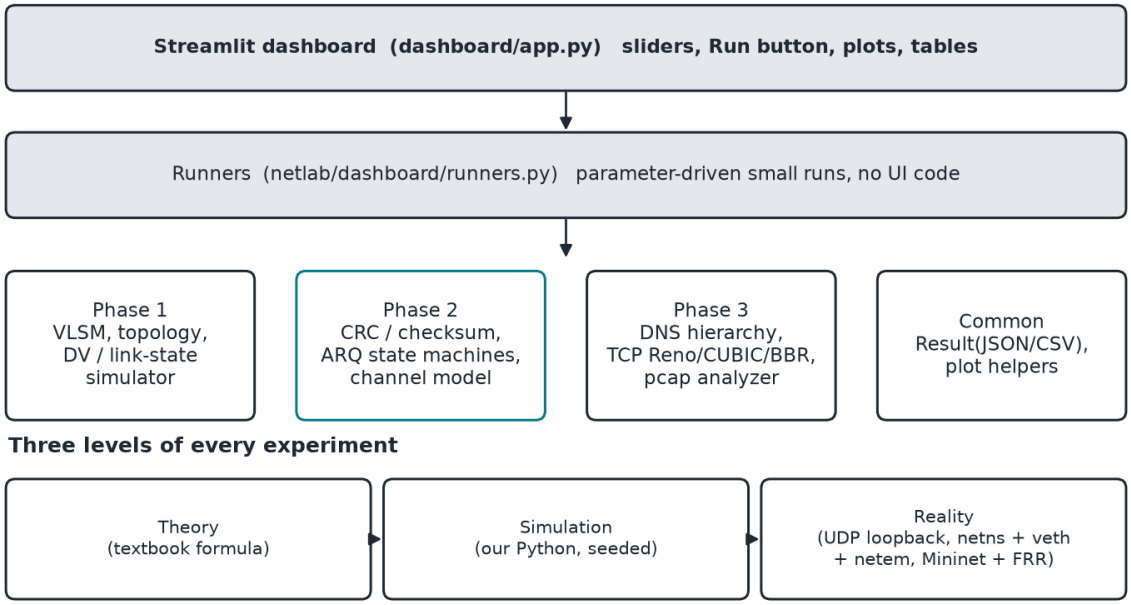


Figure 3.1: NetLab software architecture and the three evaluation levels

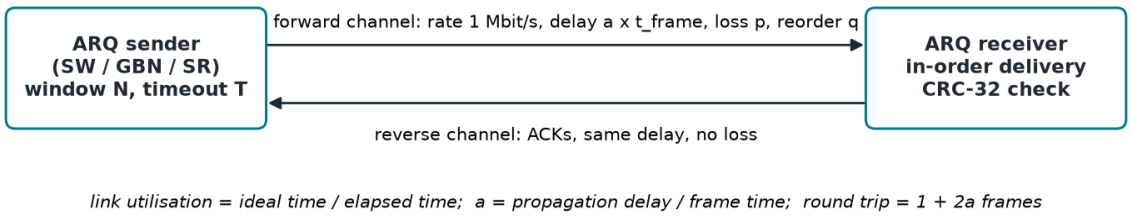


Figure 3.2: ARQ experiment topology (simulated link; the same state machines run over UDP)

3.2 Baseline mechanism description

The baseline is Selective Repeat as presented in the textbook [1]: window N = K = 1 + 2a frames, one timer per frame, timeout equal to the acknowledgement round trip, individual acknowledgements, and an N-frame receive buffer. The sender may have at most N frames

between the oldest unacknowledged frame (the window base) and the next new frame. When the base frame is acknowledged the window slides to the next unacknowledged frame.

Why K looks sufficient: in the lossless case the first frame of the window is acknowledged exactly when the sender has transmitted the K-th frame, so the window never stalls. The formula extends this to the lossy case by assuming a lost frame costs no more than its own retransmission.

3.3 Proposed mechanism: algorithm, pseudocode, flowchart

When frame s is lost, the base cannot move beyond s. The sender discovers the loss only when the timer of s expires (one round trip after the first transmission) and the retransmission is acknowledged one further round trip later. During that time the window is full if N = K, and the link idles. If N = 2K there are enough new frames to keep sending while the recovery is in progress. The proposed rule is:

```
N_proposed = ceil( 2 * (1 + 2a) )     # baseline: N = 1 + 2a
```

Algorithm 3.1 gives the sender, with the new rule on the first line. The receiver is unchanged except that its buffer must hold N_proposed frames.

```
procedure SR_SENDER(frames, a, T):          # T = timeout = ACK round trip
    N <- ceil(2 * (1 + 2a))                   # PROPOSED (baseline: 1 + 2a)
    base <- 0; next <- 0
    loop until base = len(frames):
        while next < base + N and next < len(frames):
            send(next); start_timer(next, T); next <- next + 1
        event <- wait(ACK, TIMEOUT)
        if event = ACK(s):
            mark s acked; stop_timer(s)
            while base is acked: base <- base + 1      # slide window
        if event = TIMEOUT(s):
            send(s); restart_timer(s, T)                # only frame s
```

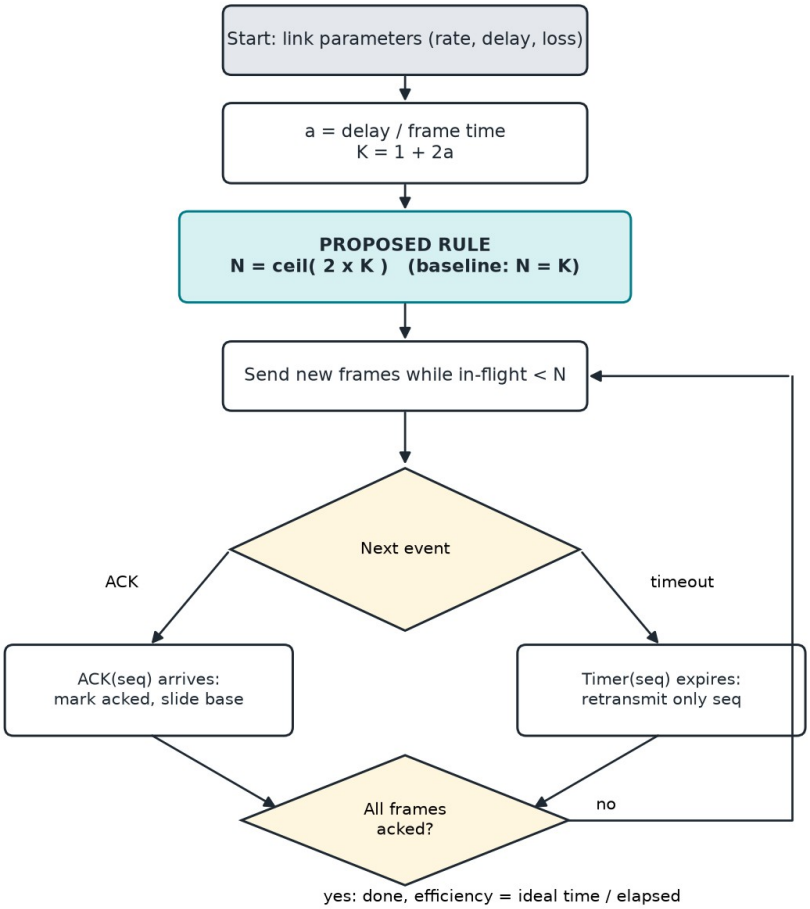


Figure 3.3: Flowchart of the proposed Selective Repeat sender

3.4 Novelty statement (what exactly differs from the baseline)

- **One line differs in the protocol:** the window is set to 2(1+2a) instead of 1+2a. Timers, acknowledgements and receiver logic are the same as the baseline, so the comparison isolates the window.
- **The rule is derived from a measured gap**, not from a model: the baseline formula predicts 0.95 at 5% loss but both the simulator (0.717 at N = 11) and real UDP (0.702) are well below it.
- **The evaluation method is new for this course setting:** the same experiment is run as theory, simulation and real sockets, with ten seeds per scenario, and reported as mean and standard deviation.

3.5 Assumptions and design decisions

- Frame loss is applied to data frames only; acknowledgements are not lost. This matches the textbook assumptions and makes the comparison fair. ACK loss is available in the channel model but is not part of the main scenarios.
- The timeout equals the acknowledgement round trip. Chapter 5.4 shows what happens with other timeouts.

- Sequence numbers are 32 bits and never wrap. This keeps the window code readable; a real protocol would use modular arithmetic and a sequence space of at least 2N.
- Frames are `kind(1) | value(4) | payload | CRC-32(4)`. A frame that fails the CRC is dropped silently, as on a real link.
- The same state machine classes (`arq.py`) run in the discrete-event simulator and over UDP sockets, so the two levels differ only in the transport.

4. IMPLEMENTATION

4.1 Tools, platform, versions and hardware

Item	Version / detail
Operating system	Linux 7.2 (Arch-based Omarchy), x86-64, 14 logical CPUs, 15 GB RAM
Language and runtime	Python 3.14, managed with uv
Libraries	matplotlib 3.11 (plots), pandas 3.0 and Streamlit 1.66 (dashboard), pytest 8 (tests)
Real-network tools	iproute2 (`ip`, `tc netem`), `unshare` for unprivileged network namespaces
Testbed (written, not run for this report)	Mininet, FRRouting (RIP, OSPF), iperf3, tcpdump on an Ubuntu VM
Network design tool	Cisco Packet Tracer (guide in docs/phase1/PACKET_TRACER.md)
Version control	Git, hosted on GitHub

Table 4.1: Software and hardware environment

4.2 Simulation or testbed setup

Three levels of setup are used for the proposed mechanism.

- **Simulation.** A discrete-event simulator (`netlab/reliability/sim.py`) drives the sender and receiver over two `Channel` objects. Each run sends 3000 frames of 109 bytes (100-byte payload + 9 bytes of header and CRC) at 1 Mbit/s.
- **Real UDP on loopback.** The same state machines run over UDP sockets with a user-space relay that adds delay, loss and reordering. Frames are 1009 bytes at 4 Mbit/s (about 2 ms per frame), so that operating-system timer jitter is a small fraction of a frame time. 150 frames per run.
- **Real kernel link.** `scripts/netns_e2e.sh` creates two network namespaces joined by a veth pair, applies `tc netem delay 10ms loss 5%` on the data direction and runs the ARQ and DNS command-line tools as separate processes, without root privileges.

Parameter	Simulation	Real UDP (loopback)
a (propagation / frame time)	5 (also 1, 10, 20 in the sensitivity study)	5
Round trip in frames, K = 1 + 2a	11	11
Baseline window N	11 (= K)	11
Proposed window N	22 (= 2K)	22
Timeout	ACK round trip (2a frame times)	ACK round trip + 1 ms
Loss, scenarios S1 / S2 / S3	1% / 5% / 20%	1% / 5% / 20%
Frames per run	3000	150
Repeated runs (seeds)	10	10

Table 4.2: Parameters of the baseline versus proposed experiment

4.3 Code structure and explanation of key modules

Module	Lines	Role
netlab/reliability/arq.py	~250	Sender and receiver state machines for SW, GBN, SR; no clock and no socket
netlab/reliability/sim.py	~100	Discrete-event driver (heap of arrival events plus the sender deadline)
netlab/reliability/channel.py	~100	Seeded channel: rate, delay, jitter, loss, corruption, duplication, reordering
netlab/reliability/udp.py	~300	Real-time driver over UDP, relay, and a send/recv command-line interface
netlab/reliability/theory.py	~60	Textbook formulas and `recommended_sr_window(a)` (the proposed rule)
experiments/phase2_sr_window.py	~120	Baseline vs proposed experiment: scenarios, sensitivity, real level
netlab/dashboard/runners.py, dashboard/app.py	~400	Parameter-driven runs and the Streamlit pages
netlab/routing, topology, addressing	~700	VLSM, Dijkstra, distance vector with split horizon and poison reverse
netlab/dns	~900	DNS wire format, authoritative servers, caching resolver, hardening
netlab/tcp, netlab/analyzer	~950	TCP Reno/CUBIC/BBR simulator; pcap reader and flow analyzer

Table 4.3: Key modules (about 6,500 lines of Python in total, 94 unit tests)

The proposed rule is implemented in one function and used by the experiment and the dashboard:

```
def recommended_sr_window(a: float) -> int:
    """RTT-aware Selective Repeat window: 2(1+2a) frames, rounded up."""
    import math
    return math.ceil(2 * (1 + 2 * a))
```

The simulator measures **efficiency** as ideal_time / elapsed, where ideal_time is the time to send all frames back to back with no loss, so 1.0 means the link carried only first-copy frames at full rate.

4.4 Challenges faced and how they were solved

Challenge	Solution
Simulation and real runs must use identical protocol logic	State machines never read a clock or a socket; a driver passes frames and the current time in (sim.py, udp.py)
Selective Repeat results disagreed with the textbook	Traced the stall: the window cannot slide until the retransmission is acknowledged. Verified with the window sweep (Figure 5.3), then proposed N = 2K
OS timer jitter on loopback makes real runs noisy	Used 2 ms frames, a 1 ms timeout margin and ten seeds, and report standard deviations

Challenge	Solution
No root or Mininet on the development machine	Built an unprivileged network-namespace test (unshare + veth + netem) as the real-kernel level
BBR fairness result looked wrong	Recognised the simplified BBR model as the cause and withdrew the result instead of reporting it
Dashboard demo must show baseline vs proposed live	Added a runner and a dashboard page with sliders for a, loss and number of runs

Table 4.4: Challenges and solutions

5. EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION

5.1 Performance metrics and why they were chosen

- **Link efficiency** (primary): ideal transmission time divided by elapsed time. It is the quantity the textbook formulas predict, it is between 0 and 1, and it directly shows idle link time.
- **Retransmissions per frame** and **out-of-order discards per frame**: explain *why* efficiency changed (wasted frames versus idle time).
- **Mean and standard deviation over repeated runs**: each run uses a different seed for the loss process, so the standard deviation shows whether a difference is larger than run-to-run noise.
- **Improvement (%)** = (proposed mean / baseline mean - 1) x 100.
- For other phases: cold and cached DNS latency, TCP throughput and RTT, Jain fairness index [14], and the fraction of undetected errors.

5.2 Scenarios and parameter values

Scenario	Loss probability	Meaning
S1 (low load)	1%	Clean link; losses are rare
S2 (medium load)	5%	Typical wireless or congested link
S3 (high load)	20%	Very lossy link; stress test

Table 5.1: Scenarios (a = 5, K = 11, baseline N = 11, proposed N = 22)

Two further studies check the limits of the proposal: a sweep of a at 5% loss (Table 5.3) and a sweep of the window size at 5% loss (Figure 5.3). The other phases use the settings listed in their sections below.

5.3 Results: tables and graphs, baseline versus proposed

Table 5.2 gives the main result. All values are efficiency, mean ± standard deviation over 10 runs.

Scenario	Metric	Baseline (mean ± SD)	Proposed (mean ± SD)	Improvement (%)
S1 (low load)	Efficiency, simulation	0.909 ± 0.010	0.914 ± 0.008	+0.6
S2 (medium load)	Efficiency, simulation	0.717 ± 0.016	0.776 ± 0.009	+8.2
S3 (high load)	Efficiency, simulation	0.478 ± 0.008	0.590 ± 0.005	+23.5
S1 (low load)	Efficiency, real UDP	0.871 ± 0.049	0.872 ± 0.049	+0.1
S2 (medium load)	Efficiency, real UDP	0.702 ± 0.062	0.754 ± 0.061	+7.4
S3 (high load)	Efficiency, real UDP	0.443 ± 0.039	0.552 ± 0.048	+24.6

Table 5.2: Selective Repeat, baseline window 11 versus proposed window 22 (a = 5)

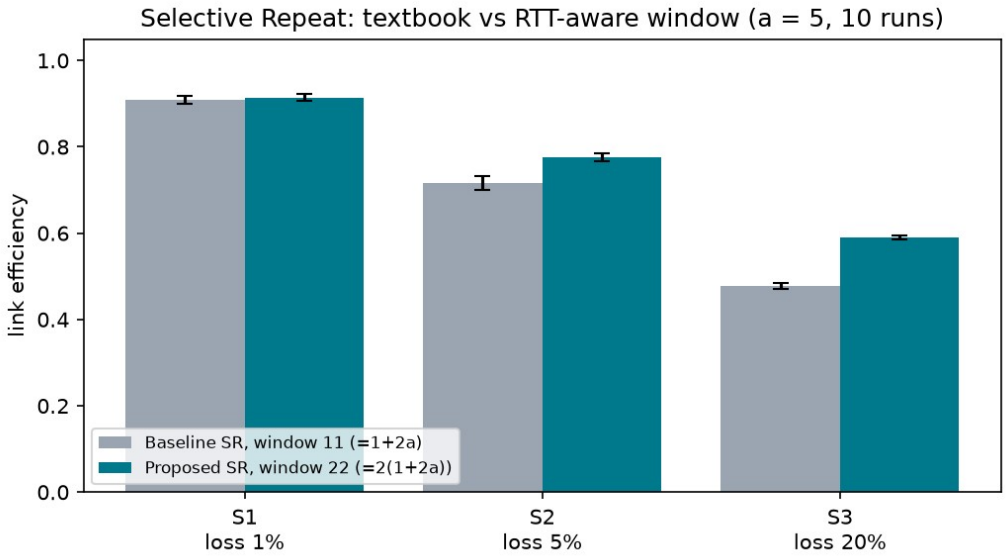


Figure 5.1: Baseline versus proposed Selective Repeat window (simulation, 10 runs, error bars = SD)

Table 5.3 shows that the benefit grows with the round-trip length, because a longer round trip means a longer stall for every loss.

a	Baseline N	Proposed N	Baseline (mean ± SD)	Proposed (mean ± SD)	Improvement (%)
1	3	6	0.873 ± 0.012	0.887 ± 0.008	+1.6
5	11	22	0.717 ± 0.016	0.776 ± 0.009	+8.2
10	21	42	0.639 ± 0.012	0.719 ± 0.012	+12.5
20	41	82	0.567 ± 0.014	0.674 ± 0.014	+18.9

Table 5.3: Effect of the round trip (a) at 5% loss, simulation, 10 runs

The remaining figures place the three levels of the other experiments on the same axes.

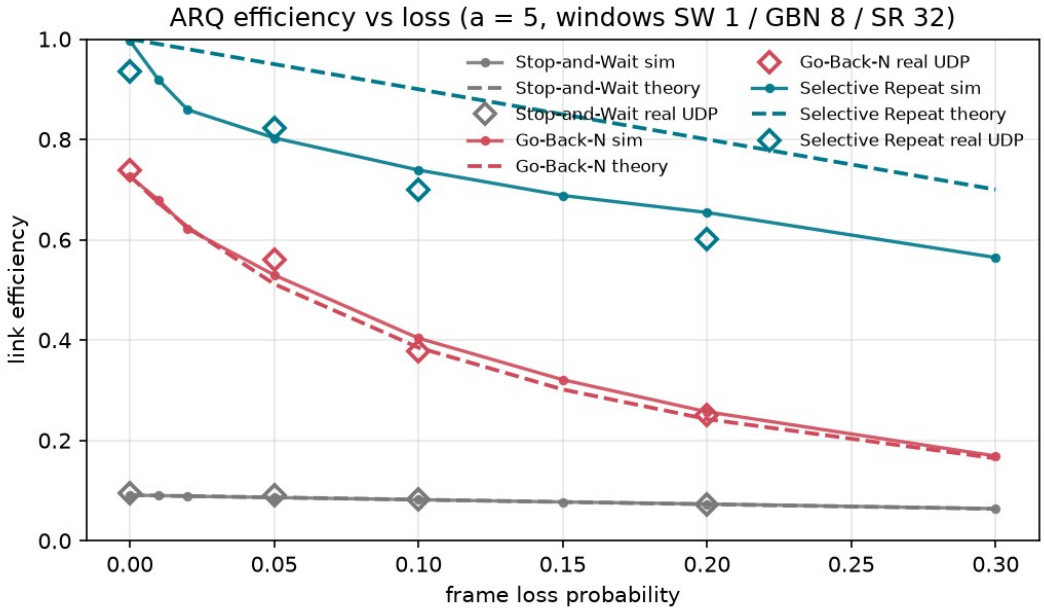


Figure 5.2: ARQ efficiency vs loss: theory, simulation and real UDP (SW 1 / GBN 8 / SR 32)

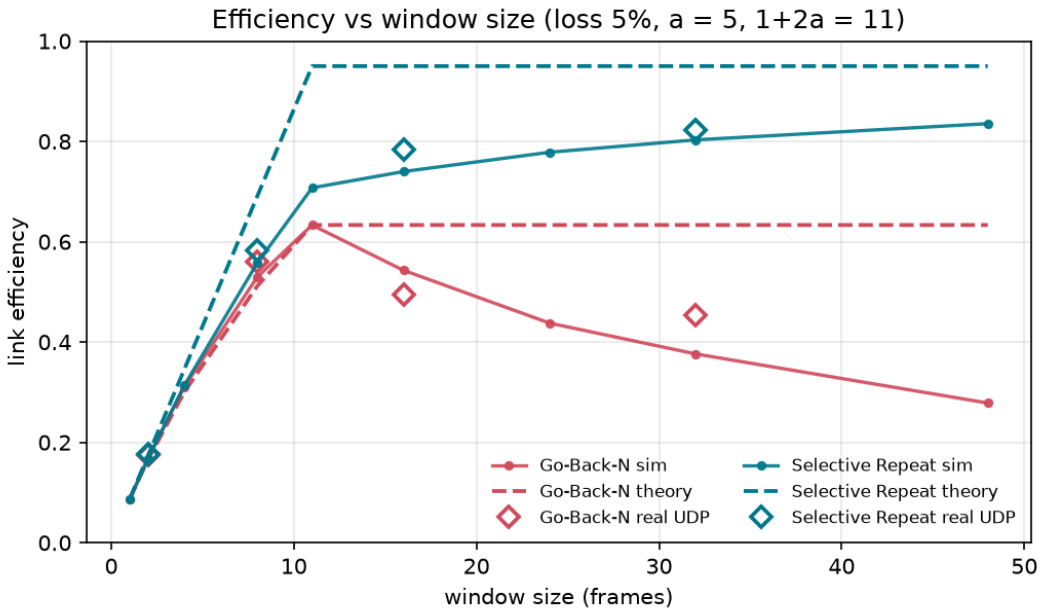


Figure 5.3: Efficiency vs window size at 5% loss; SR keeps rising beyond 1+2a = 11

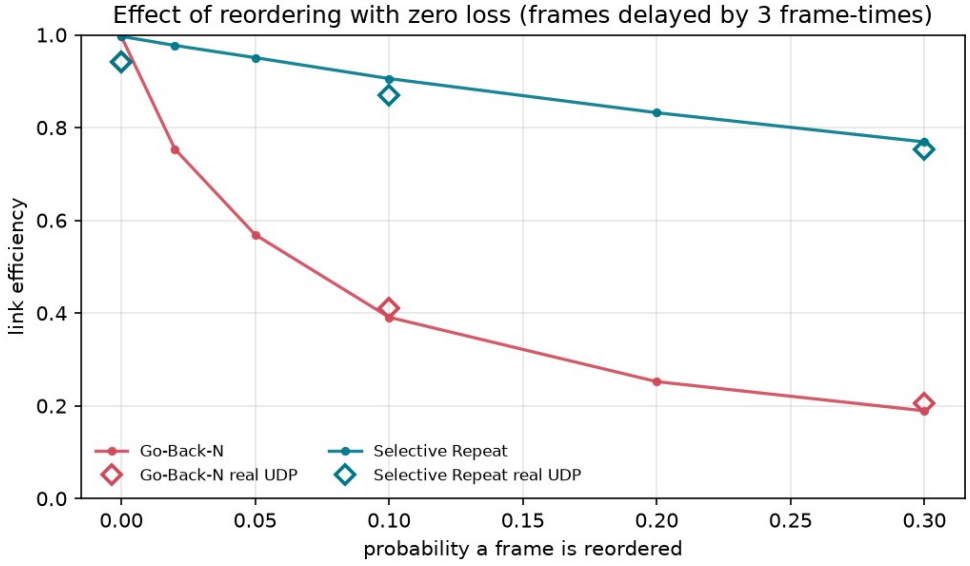


Figure 5.4: Reordering with zero loss: Go-Back-N collapses, Selective Repeat holds

Reorder probability	Go-Back-N efficiency	Selective Repeat efficiency
10%	0.39	0.91
30%	0.19	0.77

Table 5.4: Zero loss, window 16: reordering alone (simulation)

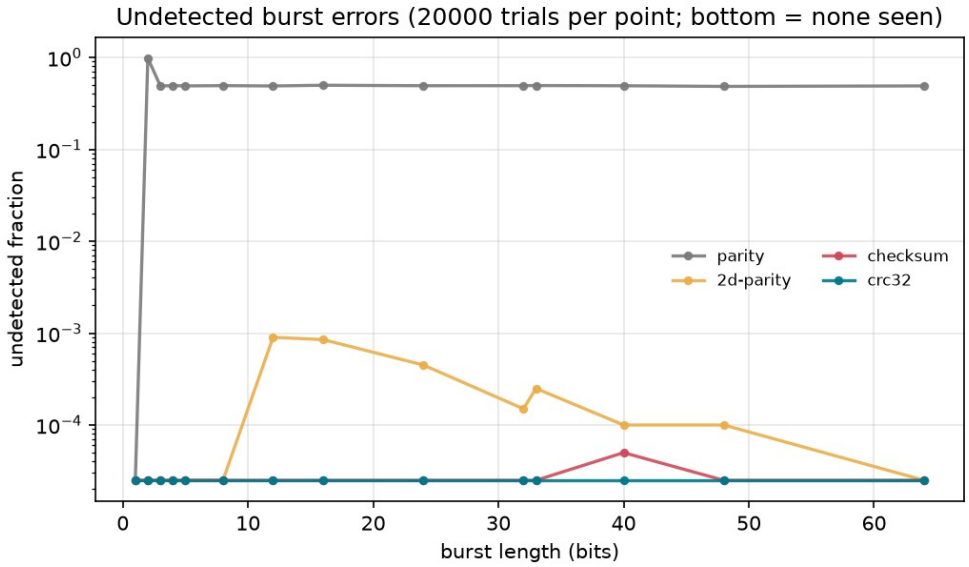


Figure 5.5: Undetected-error rate vs burst length for four detection schemes

Error	Parity	2-D parity	Internet checksum	CRC-32
single bit	caught	caught	caught	caught
2-bit burst	misses 98.6%	caught	caught	caught
bursts of 3 to 64 bits	misses ~49%	misses <= 0.09%	<= 1 in 20,000	none seen
two swapped 16-bit words	misses 100%	misses 24%	misses 100%	none seen
add to one word, subtract from another	misses 53%	misses 7%	misses 100%	none seen

Table 5.5: Undetected errors, 64-byte messages, 20,000 trials per point

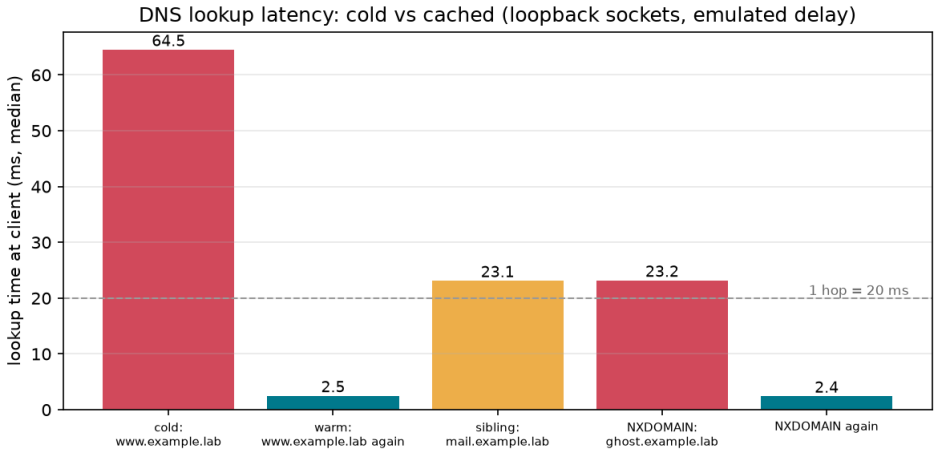


Figure 5.6: DNS latency: cold lookup, cached answer, sibling name and NXDOMAIN

DNS lookup	Median time	What happened
cold www.example.lab	64 ms	client to resolver, then root, TLD and authoritative: 3 x 20 ms + 2 ms
same name again	2.5 ms	answer cached
sibling mail.example.lab	23 ms	delegations cached, one upstream query
NXDOMAIN, cold / again	23 ms / 2.4 ms	one upstream query, then negative cache

Table 5.6: DNS latency, real UDP, each server 20 ms away (10 repeats)

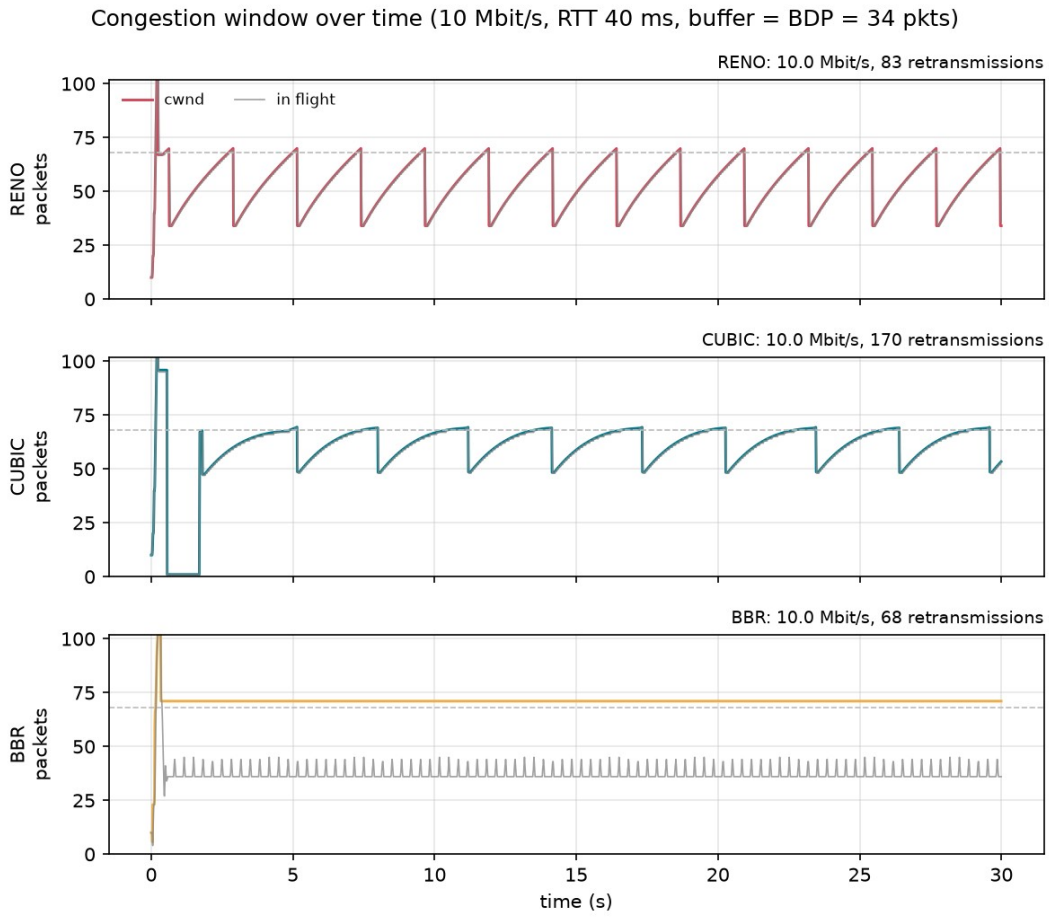


Figure 5.7: TCP congestion window of Reno, CUBIC and BBR (10 Mbit/s, 40 ms RTT, simulation)

Experiment	Result
Bottleneck throughput, one flow	Reno, CUBIC and BBR all about 10 Mbit/s
Mean RTT (floor 40 ms)	Reno 61 ms, CUBIC 70 ms, BBR 42.5 ms
Bufferbloat, Reno RTT vs buffer	44 ms (10 pkts), 61 ms (34 pkts = BDP), 219 ms (400 pkts); throughput unchanged
1% random loss	Reno 2.9, CUBIC 3.2 Mbit/s (near the Mathis model [4]); BBR 9.97 Mbit/s
Jain index, two flows	Reno-Reno 1.00, BBR-BBR 0.99, CUBIC-CUBIC 0.92; CUBIC takes ~78% against Reno
Analyzer vs simulator (Reno / CUBIC / BBR)	retransmissions 78 / 166 / 68 identical; median RTT within 1.2 ms; throughput within 0.02 Mbit/s

Table 5.7: TCP simulation and pcap analyzer validation

Routing (Phase 1). A plain distance-vector protocol counts to infinity (16) in 15 rounds on both test graphs. Split horizon with poison reverse reduces the line graph to 2 rounds but leaves the triangle at 15, since a three-router loop is invisible to it. On the trap topology, RIP (hop count) selects the 1 Mbit/s direct link while OSPF (cost = reference bandwidth / bandwidth)

selects the 3-hop 100 Mbit/s path, a predicted 100x throughput gap. This is a **prediction from the simulator**; the testbed confirmation was not run.

Real kernel link (network namespaces). One run of `make e2e-netns` (20 ms round trip, 5% loss on the data direction, 60 frames, window 16, separate processes) delivered 60/60 frames with Stop-and-Wait (efficiency 0.080, 6 retransmissions), Go-Back-N (0.868, 0 retransmissions) and Selective Repeat (0.413, 6 retransmissions). A DNS lookup of www.example.lab across the link returned 10.20.0.10 in 22.2 ms (cold) and 20.4 ms. With only 60 frames and random loss these single-run values vary from run to run (an earlier run gave 0.09 / 0.43 / 0.69), so they demonstrate that the protocols work on a real link, not precise efficiencies.

5.4 Analysis and discussion

- **Why the textbook window is too small.** With N = K, a lost frame stops the window from sliding for roughly two round trips: one until the timer fires, one until the retransmission is acknowledged. The formula counts only the retransmission. Doubling the window supplies the missing new frames, which is why the gain is largest where loss is highest (+23.5% at 20%) and negligible where it is lowest (+0.6% at 1%).
- **Trend with a.** The improvement rises from +1.6% (a = 1) to +18.9% (a = 20): the longer the path, the longer each stall, and the more the doubled window helps.
- **Simulation and reality agree.** Real UDP gives +0.1%, +7.4% and +24.6% against +0.6%, +8.2% and +23.5%. The real standard deviations are about five times larger (150 frames per run instead of 3000, plus OS jitter), and in S1 and S2 the real baseline and proposed means differ by less than one SD, so only S3 is a statistically clear real-level win; the simulation, with far smaller SD, resolves S2 clearly.
- **Other findings.** Stop-and-Wait matches 1/(1+2a) almost exactly. Go-Back-N matches its formula within a few percent and collapses under loss and under reordering. CRC-32 is the only scheme that caught all adversarial edits; the Internet checksum is blind to any change that preserves the sum. A cold DNS lookup is about 26 times slower than a cached one, as the hop arithmetic predicts. Loss-based TCP fills the buffer and pays in RTT, while BBR does not.
- **Timeout trade-off.** Timeouts shorter than the round trip cause spurious retransmissions and a sharp efficiency drop; longer timeouts idle the link for longer after each real loss (arq_timeout.png).

5.5 Critical evaluation: strengths, weaknesses, conditions where the proposed scheme fails

Strengths	Weaknesses and failure conditions
One-line change; protocol logic otherwise identical, so the comparison is clean	Does **not** reach the textbook ideal: at 5% loss the proposed window gives 0.776 against a predicted 0.95. A larger window helps further (0.835 at N = 48) with diminishing returns, so 2K is a practical choice, not an optimum
Benefit measured with repeated runs and confirmed at the real level	No benefit on a clean link (+0.6% at 1% loss, +0.1% real), so it is pointless where loss is rare
Gain grows with round-trip length and loss rate, the cases where ARQ matters most	Doubles the sender and receiver buffer and the number of timers; fails when memory or sequence space is limited (the sequence space must be at least 2N)
Three-level method exposed the gap and can test other rules	Assumes ACKs are never lost and a timeout equal to the round trip; with ACK loss or jittery RTT the optimal window may differ and was not studied
All code, data and a live dashboard comparison are provided	Real level is loopback with a user-space delay emulator and a few runs over a namespace link; the Mininet testbed and Linux TCP runs are not done

Table 5.8: Critical evaluation

6. CONCLUSION AND FUTURE WORK

6.1 Summary of work and key findings

NetLab implements the main protocols of the course from scratch and evaluates each one at the theory, simulation and real level. The quantified findings are:

- Selective Repeat with the textbook window (1+2a) falls below its formula under loss: 0.717 against 0.95 at 5% loss.
- The proposed RTT-aware window 2(1+2a) raises efficiency by **+0.6%, +8.2% and +23.5%** at 1%, 5% and 20% loss in simulation (10 runs), and by +0.1%, +7.4% and +24.6% over real UDP sockets.
- The gain grows with the round trip: +1.6% at a = 1 to +18.9% at a = 20.
- Go-Back-N falls to 0.39 efficiency at 10% reordering with no loss at all, while Selective Repeat holds 0.91.
- CRC-32 caught every error in all tests, while the Internet checksum missed 100% of reordered words; a cold DNS lookup (64 ms) is 26 times slower than a cached one (2.5 ms); and our pcap analyzer reproduces the simulator's retransmission counts exactly.
- 94 unit tests pass; `make sim`, `make e2e` and `make e2e-netns` reproduce the results.

6.2 Limitations

- The Mininet + FRR and Linux TCP (iperf3) experiments were not run, so RIP versus OSPF and Reno/CUBIC/BBR results are simulation only.
- The proposed window is an empirical rule validated for ACKs without loss and a timeout equal to the round trip; it does not reach the theoretical ideal.
- The BBR model is simplified and its fairness result is not reported as a finding.
- Real-level ARQ results come from loopback with a user-space relay, and have large standard deviations because each run has only 150 frames.

6.3 Future enhancements

- Run the Mininet + FRR and iperf3 experiments on the Ubuntu testbed and add their plots.
- Study the window rule with ACK loss, jitter and adaptive timeouts; derive the window analytically from the stall time instead of the factor 2.
- Search for the optimal window numerically over (a, p) and compare with 2K.
- Complete the BBR model (ProbeRTT, loss response) and compare with Linux BBR.
- Extend the real level to longer runs in network namespaces with `tc netem`, with repeated runs.

7. REFERENCES

References (IEEE style)

[1] W. Stallings, Data and Computer Communications, 10th ed. Upper Saddle River, NJ, USA: Pearson, 2013.

[2] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed. Boston, MA, USA: Pearson, 2021.

[3] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in Proc. 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX), 2010.

[4] M. Mathis, J. Semke, J. Mahdavi, and T. Ott, "The macroscopic behavior of the TCP congestion avoidance algorithm," ACM SIGCOMM Computer Communication Review, vol. 27, no. 3, pp. 67-82, 1997.

[5] S. Ha, I. Rhee, and L. Xu, "CUBIC: A new TCP-friendly high-speed TCP variant," ACM SIGOPS Operating Systems Review, vol. 42, no. 5, pp. 64-74, 2008.

[6] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-based congestion control," ACM Queue, vol. 14, no. 5, pp. 20-53, 2016.

[7] J. Gettys and K. Nichols, "Bufferbloat: Dark buffers in the Internet," Communications of the ACM, vol. 55, no. 1, pp. 57-65, 2012.

[8] R. Braden, D. Borman, and C. Partridge, "Computing the Internet checksum," RFC 1071, IETF, 1988.

[9] P. Koopman and T. Chakravarty, "Cyclic redundancy code (CRC) polynomial selection for embedded networks," in Proc. Int. Conf. Dependable Systems and Networks (DSN), 2004, pp. 145-154.

[10] G. Malkin, "RIP Version 2," RFC 2453, IETF, 1998.

[11] J. Moy, "OSPF Version 2," RFC 2328, IETF, 1998.

[12] P. Mockapetris, "Domain names - concepts and facilities," RFC 1034, and "Domain names - implementation and specification," RFC 1035, IETF, 1987.

[13] M. Allman, V. Paxson, and E. Blanton, "TCP congestion control," RFC 5681, IETF, 2009.

[14] R. Jain, D.-M. Chiu, and W. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," DEC Research Report TR-301, 1984.

[15] A. Hubert and R. van Mook, "Measures for making DNS more resilient against forged answers," RFC 5452, IETF, 2009.

APPENDIX A: IMPORTANT SOURCE CODE OR CONFIGURATION LISTINGS

A.1 Proposed rule (netlab/reliability/theory.py)

```
def eff_selective_repeat(a: float, p: float, n: int) -> float:
    k = 1 + 2 * a
    return (1 - p) if n >= k else n * (1 - p) / k      # textbook model (baseline)



def recommended_sr_window(a: float) -> int:
    """RTT-aware Selective Repeat window: 2(1+2a) frames, rounded up."""
    import math
    return math.ceil(2 * (1 + 2 * a))                  # proposed rule
```

A.2 Repeated-run comparison (experiments/phase2_sr_window.py)

```
def runs(window, loss, a, n, seeds):
    _, delay, timeout = standard_setup(a, FRAME)       # timeout = ACK round trip
    out = []
    for s in range(1, seeds + 1):
        r = simulate("sr", n, window, timeout,
                     Channel(delay=delay, loss=loss, rate_bps=1e6, seed=s),
                     Channel(delay=delay, seed=1000 + s), PAYLOAD)
        assert r.finished and r.wrong_payloads == 0
        out.append(r.efficiency)
    return out
# per scenario: baseline = runs(11, p, ...), proposed = runs(22, p, ...); report
mean and stdev
```

A.3 Commands to reproduce

```
uv sync --extra dashboard
make test                                              # 94 unit tests
uv run python -m experiments.phase2_sr_window --real   # Tables 5.2 and 5.3
make sim                                               # every no-root experiment
make e2e-netns                                         # ARQ + DNS over veth + netem
make dashboard                                         # http://localhost:8501
```

A.4 Network-namespace link (scripts/netns_e2e.sh, excerpt)

```
ip netns add h1; ip netns add h2
ip link add v1 type veth peer name v2
ip netns exec h1 tc qdisc add dev v1 root netem delay 10ms loss 5%
ip netns exec h2 tc qdisc add dev v2 root netem delay 10ms
```

APPENDIX B: ADDITIONAL SCREENSHOTS, PACKET CAPTURES, RAW RESULT TABLES

B.1 Dashboard screenshots

The Streamlit dashboard (`make dashboard`) exposes each experiment with sliders. Figure B.1 is the live baseline-versus-proposed page used for the demo. All screenshots are in the repository under `screenshots/`.

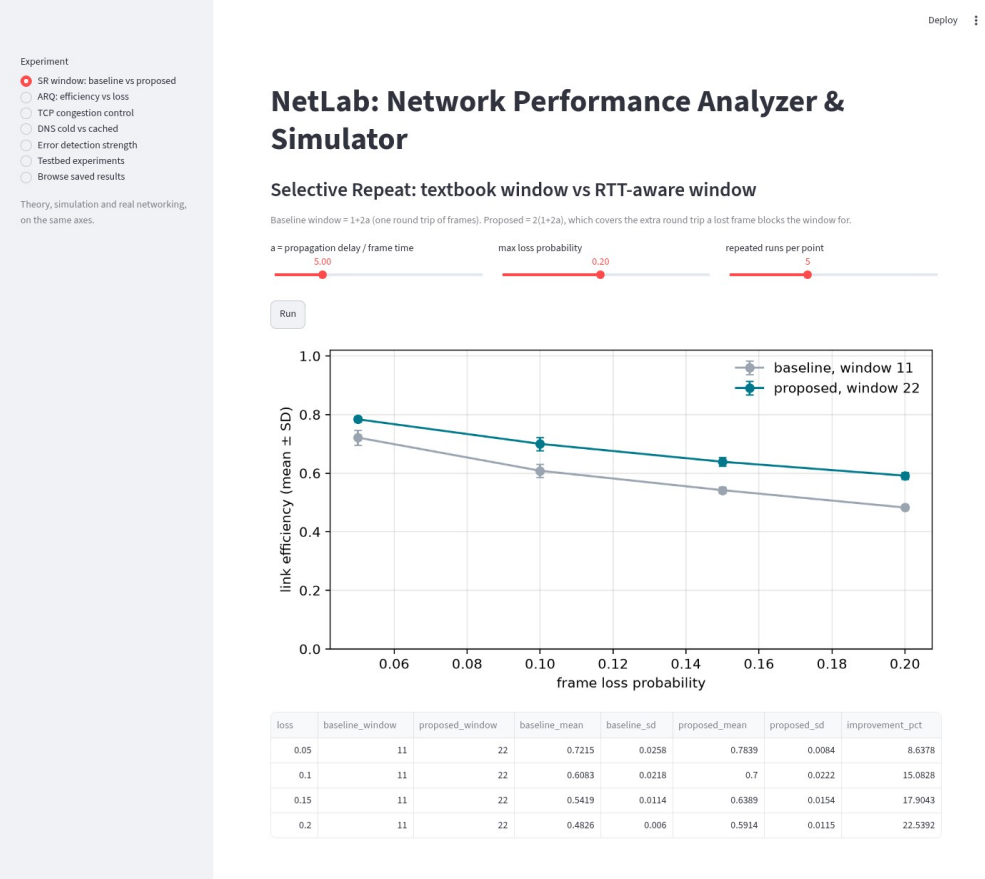


Figure B.1: Dashboard, Selective Repeat baseline vs proposed (live)

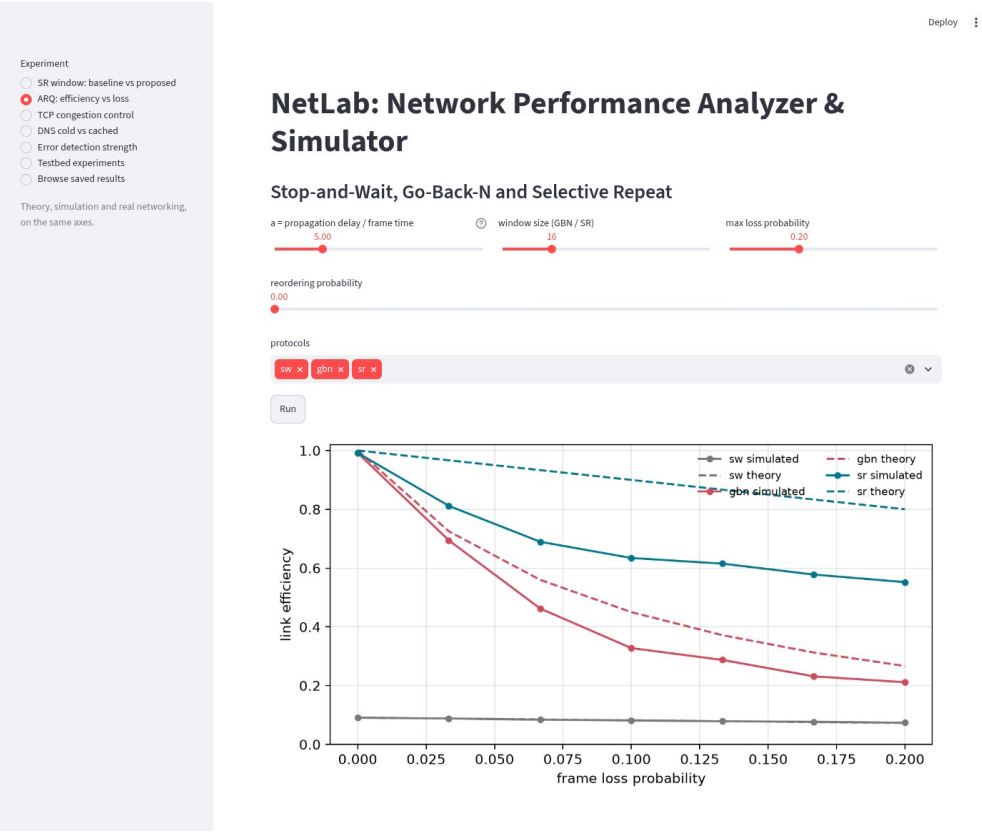


Figure B.2: Dashboard, ARQ efficiency vs loss

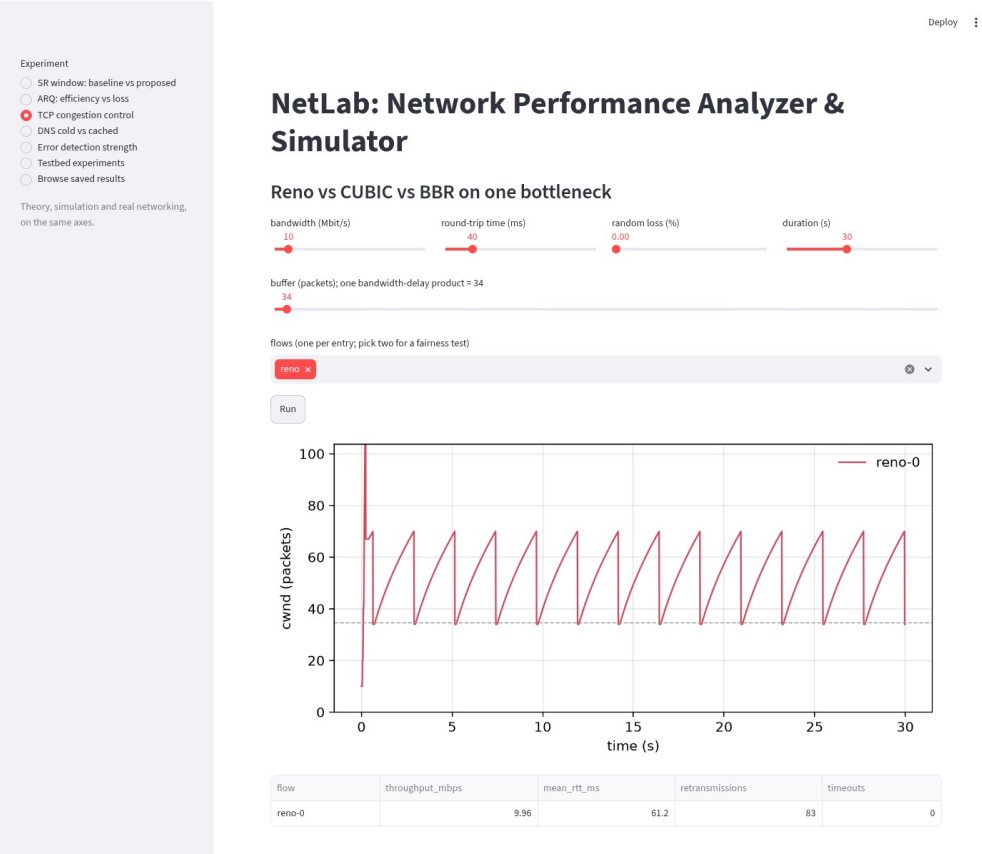


Figure B.3: Dashboard, TCP congestion control

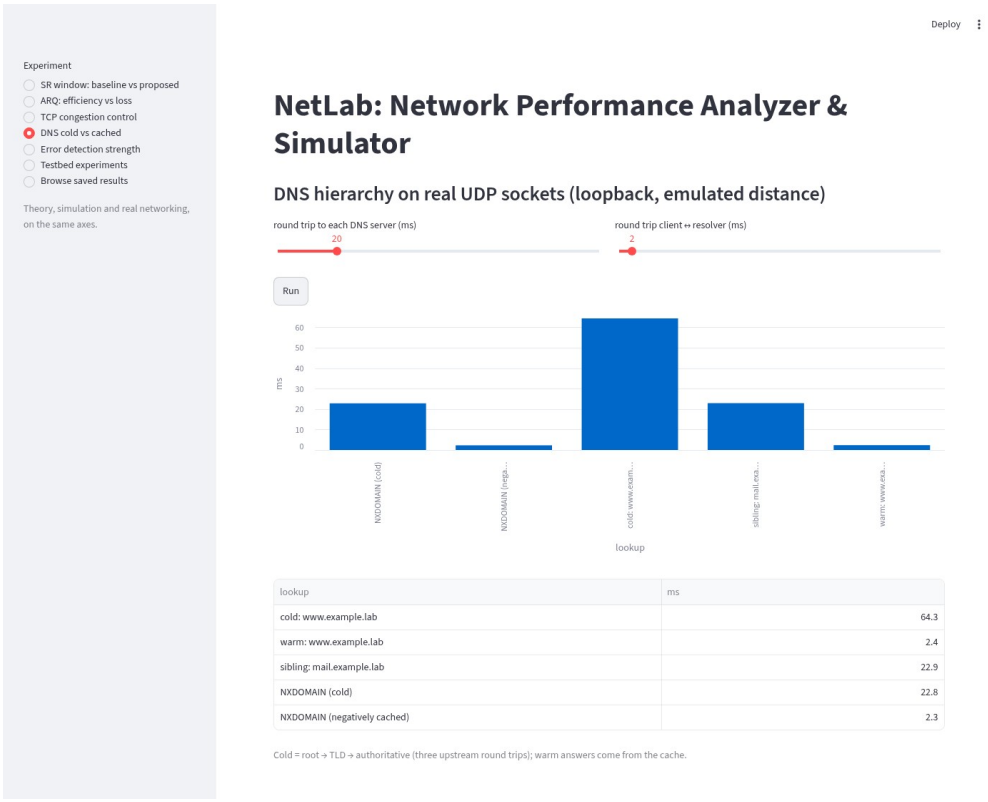


Figure B.4: Dashboard, DNS cold vs cached

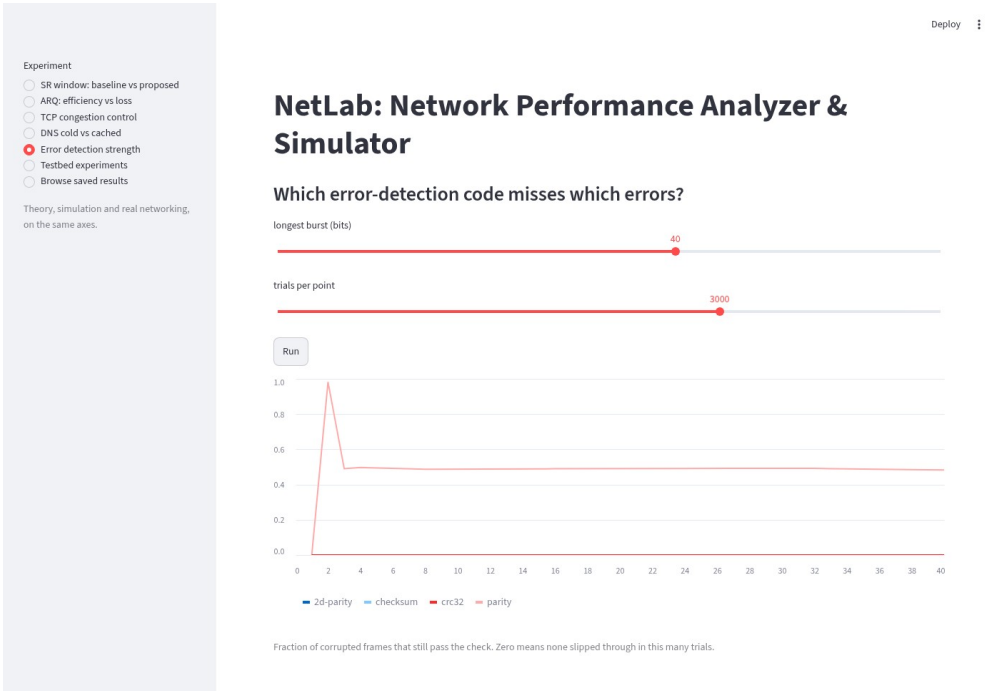


Figure B.5: Dashboard, error detection strength

B.2 Additional plots

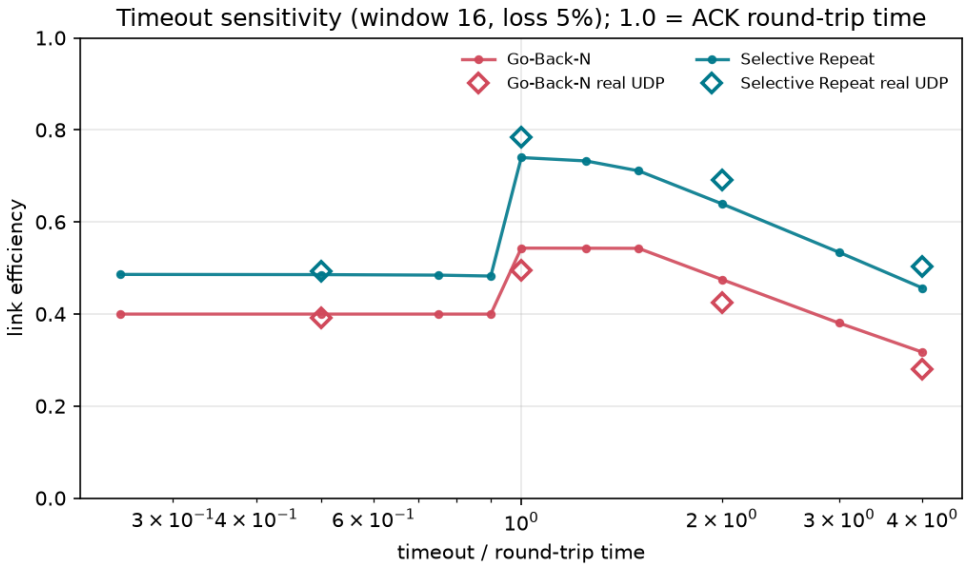


Figure B.6: Timeout sensitivity (window 16, 5% loss)

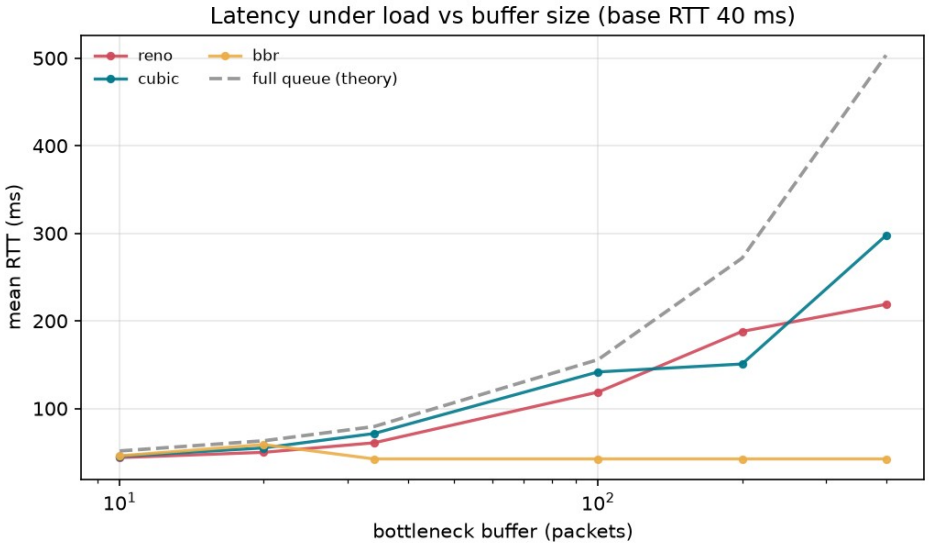


Figure B.7: Bufferbloat, RTT vs buffer size

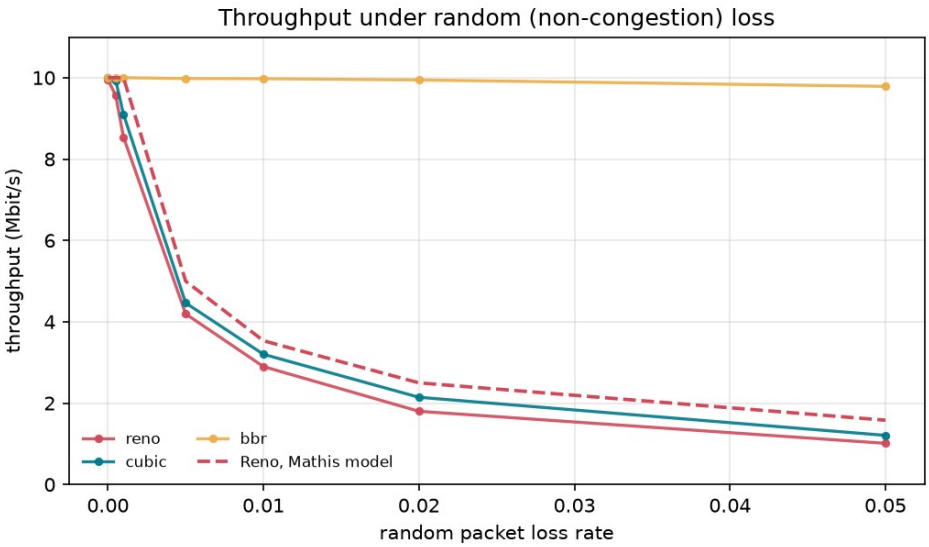


Figure B.8: TCP throughput vs random loss

B.3 Raw output of the namespace test (make e2e-netns)

```
3 packets transmitted, 2 received, 33.3333% packet loss, time 404ms
rtt min/avg/max/mdev = 20.032/20.039/20.047/0.007 ms
  ok   link up, ping through netem
  ok   ARQ sw over veth+netem: 60/60 delivered, efficiency 0.080, retransmissions 6
  ok   ARQ gbn over veth+netem: 60/60 delivered, efficiency 0.868, retransmissions
0
  ok   ARQ sr over veth+netem: 60/60 delivered, efficiency 0.413, retransmissions 6
  ok   DNS across netns: [{"name": "www.example.lab", "rtt_ms": 22.206, "rcode": 0,
"answers": ["10.20.0.10"]}, {"name": "www.example.lab", "rtt_ms": 20.386, "rcode":
0, "answers": ["10...
NETNS E2E PASSED
```

B.4 Raw result files

Every experiment writes JSON, CSV and PNG to `results/<phase>/`. The tables in this report come from `sr_window_novelty.json`, `sr_window_vs_a.json`, `sr_window_real.json`, `arq_efficiency.json`, `detection.json`, `dns_latency.json`, `tcp_*.json` and `analyzer_validation.json`.